29th April - 3rd May 2026

Reclaim

Deployment Plan

From receipt to reimbursement
— in minutes, not days

Prepared for

UM Hackathon 2026
Judges

Prepared by

Team bogosort

UM Hackathon 2026 PEKOM UNIVERSITY OF MALAYA YTL AI LABS Z.AI

Table of Content

Document Control **3**
1. Document Overview **3**
1.1 Background 3
1.2 Objectives 3
1.3 Scope 4
2. Roles & Responsibilities **5**
3. Target Architecture & Environments **5**
3.1 Infrastructure 5
3.2 Dependencies 6
3.3 Configuration Management 7
4. What We Built & Why: Current Production Implementation **8**
4.1 Containerized Stack Deployed via Dokploy 8
4.2 Continuous Deployment via GitHub SSH Deploy Key + Autodeploy Webhook 9
4.3 Runtime Secret Management via Dokploy Environment UI 10
4.4 Cloudflare Proxied DNS (Orange Cloud WAF) 10
4.5 UFW Firewall: Cloudflare-Only Inbound Traffic 11
4.6 Application-Level Rate Limiting via slowapi 11
4.7 AI Observability via LangSmith 11
5. Deployment Strategy **12**
5.1 Methodology 12
5.2 CI/CD Integration 13
6. Pre-Deployment Checklist **13**
6.1 Infrastructure Readiness 13
6.2 Secret Verification 14
6.3 Data Preservation 14
6.4 Migration Testing 14
6.5 API Quota Check 14
6.6 Code Readiness 14
7. Future Architecture Roadmap **15**
7.1 Distributed Redis-Based Rate Limiting 15
7.2 Automated PostgreSQL Volume Backups to S3/MinIO 15
7.3 High Availability via Multi-Node Docker Swarm 16
7.4 Dynamic Secret Management via HashiCorp Vault 16
7.5 Self-Hosted Agent Tracing via Langfuse 17

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
Team Name	*bogosort*
Team Repo URL	https://github.com/zsh-sage/reclaim
Live Deployment URL	https://reclaimai.dev

1. Document Overview

1.1 Background

Reclaim is an AI-native expense reimbursement platform built for UM Hackathon 2026. The platform automates compliance evaluation of employee expense claims using three LangGraph agentic pipelines (Policy Agent, Document Agent, Compliance Agent). It provides two user-facing portals: an Employee Portal for claim submission and an HR Portal for policy management, triage, and final decisions.

This deployment plan covers the initial production release of the full Reclaim stack — FastAPI backend, Next.js frontend, and PostgreSQL+pgvector database — onto a self-managed VPS orchestrated via Dokploy.

1.2 Objectives

A successful deployment is defined by the following outcomes:

- All three containerized services (database, backend, frontend) pass their health checks and remain stable.
- The `/health` endpoint returns `{"status": "healthy", "database": "connected"}`.

© UMHackathon 2026

- Zero application downtime during updates via Dokploy's rolling container replacement.
- All runtime secrets are injected via Dokploy's Environment UI — no secrets committed to version control.
- The Cloudflare-proxied domain is live and HTTPS-terminated.
- LangSmith tracing is active and is capturing agent execution traces.
- UFW firewall is configured to accept traffic exclusively from Cloudflare IP ranges on ports 80/443.
- Application-level rate limiting is active on all AI-heavy endpoints.

1.3 Scope

In scope:

- PostgreSQL 16 + pgvector database container (`ankane/pgvector:v0.5.1`)
- FastAPI backend container (Python 3.13, `uv.Dockerfile`)
- Next.js 16 frontend container (Node 22 Alpine, multi-stage build)
- Dokploy deployment configuration and Autodeploy webhook
- Cloudflare DNS proxying and SSL
- UFW firewall hardening
- Application-level rate limiting via `slowapi`
- LangSmith observability integration

Out of scope for this release:

- Distributed Redis-based rate limiting
- Automated PostgreSQL volume backups to S3/MinIO
- Multi-node Docker Swarm high-availability cluster
- HashiCorp Vault secret management

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Roles & Responsibilities

Role	Name	Responsibility
Deployment Lead	Filbert Ng	Executes all deployment steps, monitors rollout, triggers rollback if needed
QA / Reviewer	Filbert Ng	Signs off on pre-deployment checklist, runs post-deployment functional tests
Approver	Darrell Benedict Setiawan	Provides final go/no-go decision before traffic is routed to the new release

3. Target Architecture & Environments

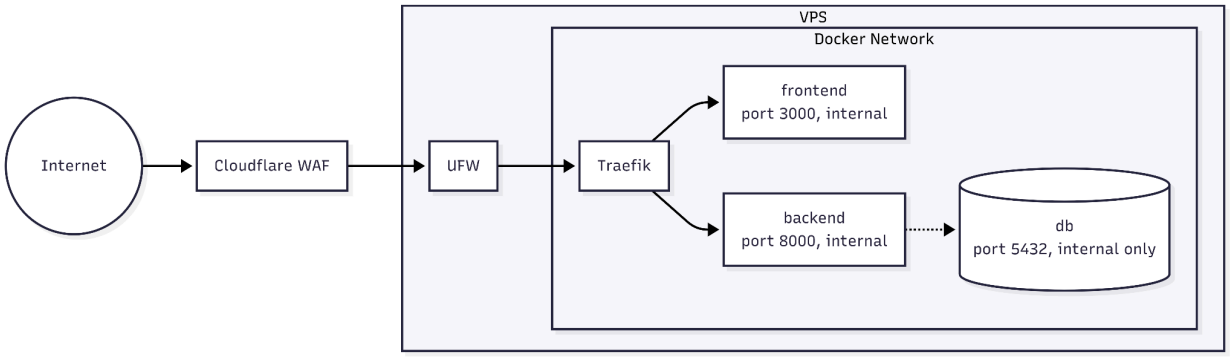
3.1 Infrastructure

Component	Specification
VPS Host	Digital Ocean Droplet, 2vCPU / 4 GB RAM, Singapore Server
OS	Ubuntu 24.04.3 LTS
Orchestration	Dokploy (Self-hosted, running on VPS)
Reverse Proxy / SSL	Traefik (managed by Dokploy) + Cloudflare WAF (Orange Cloud DNS Proxy)
Domain	reclaimai.dev

© UMHackathon 2026

Email: umhackathon@um.edu.my

Service topology (production):



In production, no container ports are exposed directly to the host (`expose` only). All inbound HTTP/HTTPS traffic arrives via Cloudflare → Traefik. The database port is never accessible from outside the Docker network.

3.2 Dependencies

Service	Provider	Purpose
Primary LLM (text)	Ilmu → ilmu-glm-5.1	Primary reasoning for compliance and policy agents
Backup LLM (text)	OpenRouter → google/gemini-3.1-flash-lite-preview	Backup model if primary LLM (GLM 5.1) unavailable
LLM (vision)	OpenRouter → qwen/qwen3.5-flash-02-23	OCR / receipt image extraction
Embeddings	OpenRouter → openai/text-embedding-3-small	Policy section vector embeddings (1536-dim, stored in pgvector)
Observability	LangSmith	LangGraph agent trace capture, token usage monitoring
DNS / WAF	Cloudflare	DDoS protection, WAF, HTTPS termination at edge

© UMHackathon 2026

3.3 Configuration Management

All runtime secrets and environment-specific values are injected at deploy time through Dokploy's Environment UI — never stored in .env files committed to the repository. The following variables must be set before deployment:

Backend environment variables:

Variable	Description
`DATABASE_URL`	PostgreSQL connection string (`postgresql://user:pass@db:5432/smart_reimburse`)
`SECRET_KEY`	JWT signing secret
`LLM_API_KEY`	ILMU GLM 5.1 API token for LLM calls
`OPENROUTER_API_KEY`	OpenRouter API key for Vision LLM and embedding calls
`CHAT_MODEL`	Primary LLM model ID (e.g., `ilmu-glm-5.1`)
`VISION_MODEL`	Vision model ID for OCR (e.g., `qwen/qwen3.5-flash-02-23`)
`EMBEDDING_MODEL`	Embedding model ID (e.g., `openai/text-embedding-3-small`)
`LLM_BASE_URL`	ILMU API base URL
`OPENROUTER_BASE_URL`	OpenRouter API base URL
`FRONTEND_URL`	Production frontend URL for CORS allow-origin
`LANGSMITH_TRACING_V2`	`true` to enable LangSmith tracing
`LANGSMITH_ENDPOINT`	LangSmith ingestion endpoint
`LANGSMITH_API_KEY`	LangSmith API key
`LANGSMITH_PROJECT`	LangSmith project name
`XENDIT_API_KEY`	Optional: Testing Payment Gateway for Payouts
`XENDIT_WEBHOOK_TOKEN`	Optional: Testing Payment Gateway for Payouts

© UMHackathon 2026

Frontend environment variables:

Variable	Description
`NEXT_PUBLIC_API_URL`	https://reclaimai.dev Browser-accessible backend URL (used by client-side fetch)
`INTERNAL_API_URL`	Container-internal backend URL (http://backend:8000) for server-side requests

4. What We Built & Why: Current Production Implementation

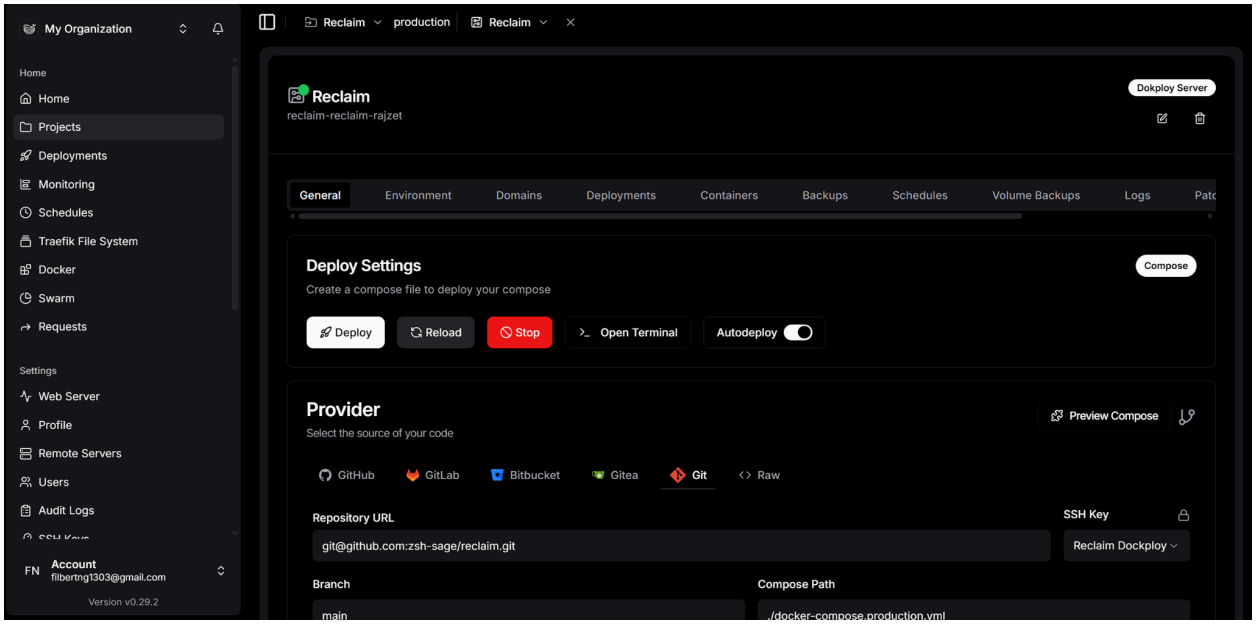
4.1 Containerized Stack Deployed via Dokploy

What We Built: All three application services (FastAPI backend, Next.js frontend, and PostgreSQL+pgvector database) are containerized using Docker and orchestrated through a production-grade `docker-compose.production.yml`. The stack is hosted on a self-managed VPS and deployed via Dokploy, a lightweight self-hosted deployment platform built on Docker Swarm.

Engineering Rationale: Dokploy operates at dramatically lower resource overhead (~350 MB RAM), preserving critical headroom for PostgreSQL's buffer pools and the parallel LangGraph agent execution that is central to Reclaim's AI pipeline. Compared to managing raw Docker commands on a Linux terminal, Dokploy provides a human-maintainable dashboard with one-click visual monitoring, centralized log aggregation, easy SSL certificate provisioning via Traefik, and native Docker Swarm support.

In production, no container ports are bound to the host machine (all services use `expose` rather than `ports`), meaning the database and backend are never reachable from the public internet, only through Traefik's internal routing.

© UMHackathon 2026



Reclaim Dokploy Dashboard Screenshot

4.2 Continuous Deployment via GitHub SSH Deploy Key + Autodeploy Webhook

What We Built: Dokploy is connected to the GitHub repository via a dedicated, read-only SSH Deploy Key. Dokploy's Autodeploy feature is enabled, so every push to the `main` branch triggers a webhook that automatically rebuilds the Docker images and performs a rolling, zero-downtime container replacement of the live services.

Engineering Rationale: Manual deployments are a primary source of human error — wrong environment variables, missed migration steps, or stale containers running outdated images. By establishing a Continuous Deployment (CD) pipeline where `git push` is the single deployment primitive, we eliminate the entire class of "forgot to redeploy" failures. The read-only deploy key is a least-privilege security boundary: it can pull code but cannot push, create branches, or alter repository state, so a compromised VPS cannot tamper with the source of truth.

The rolling update strategy means Dokploy starts the new container, waits for its health check to pass, and only then terminates the old one — preserving availability through the transition.

© UMHackathon 2026

4.3 Runtime Secret Management via Dokploy Environment UI

What We Built: All sensitive runtime credentials — PostgreSQL credentials, OpenRouter API keys, LangSmith API key, and the JWT `SECRET_KEY` — are injected exclusively through Dokploy's Environment UI. No `.env` files containing real values are committed to the repository. The codebase only contains `.env.example` files with placeholder values.

Engineering Rationale: Credentials committed to a Git repository are permanently exposed in history, even if later deleted. By keeping all secrets in Dokploy's internal environment store, the credentials exist only in the server's runtime memory and are never serialized into version control. This is the minimum viable secret management posture for any production system.

4.4 Cloudflare Proxied DNS (Orange Cloud WAF)

What We Built: The production domain's DNS A record is set to Proxied (Cloudflare's "orange cloud" mode), routing all inbound HTTP/HTTPS traffic through Cloudflare's global network before it reaches the VPS.

Engineering Rationale: This provides a frontline Web Application Firewall (WAF) at zero marginal cost. Cloudflare absorbs volumetric DDoS attacks, filters known malicious traffic patterns, and terminates HTTPS at the edge — all before a single packet reaches our server. For a hackathon project with limited backend resources, this is especially critical: a DDoS that would saturate the VPS is absorbed by Cloudflare's distributed infrastructure instead.

Type	Name	Content	Proxy status	TTL	Actions
A	dokploy	[redacted]	Proxied	Auto	Edit
A	reclaimai.dev	[redacted]	Proxied	Auto	Edit
A	www	[redacted]	Proxied	Auto	Edit

Reclaim's Cloudfare DNS Record Screenshot

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.5 UFW Firewall: Cloudflare-Only Inbound Traffic

What We Built: The VPS firewall (UFW — Uncomplicated Firewall) is configured to deny all direct public traffic on ports 80 and 443, with explicit allow rules for each IP range published in Cloudflare's official IP list. All other inbound traffic to those ports is dropped at the kernel level.

Engineering Rationale: Cloudflare's WAF only protects traffic that flows through it. Without this firewall rule, a sufficiently motivated attacker who discovers the VPS's raw IP address can bypass Cloudflare entirely by sending HTTP requests directly to the server — completely circumventing DDoS protection, WAF filtering, and rate limiting. This UFW rule enforces a zero-trust network boundary: the only path into the application is through Cloudflare.

4.6 Application-Level Rate Limiting via slowapi

What We Built: Strict per-user rate limits are enforced at the FastAPI application layer using slowapi on all AI-intensive endpoints (`/api/v1/reimbursements`, `/api/v1/documents`, `/api/v1/policies`). Requests exceeding the configured threshold receive an HTTP `429 Too Many Requests` response.

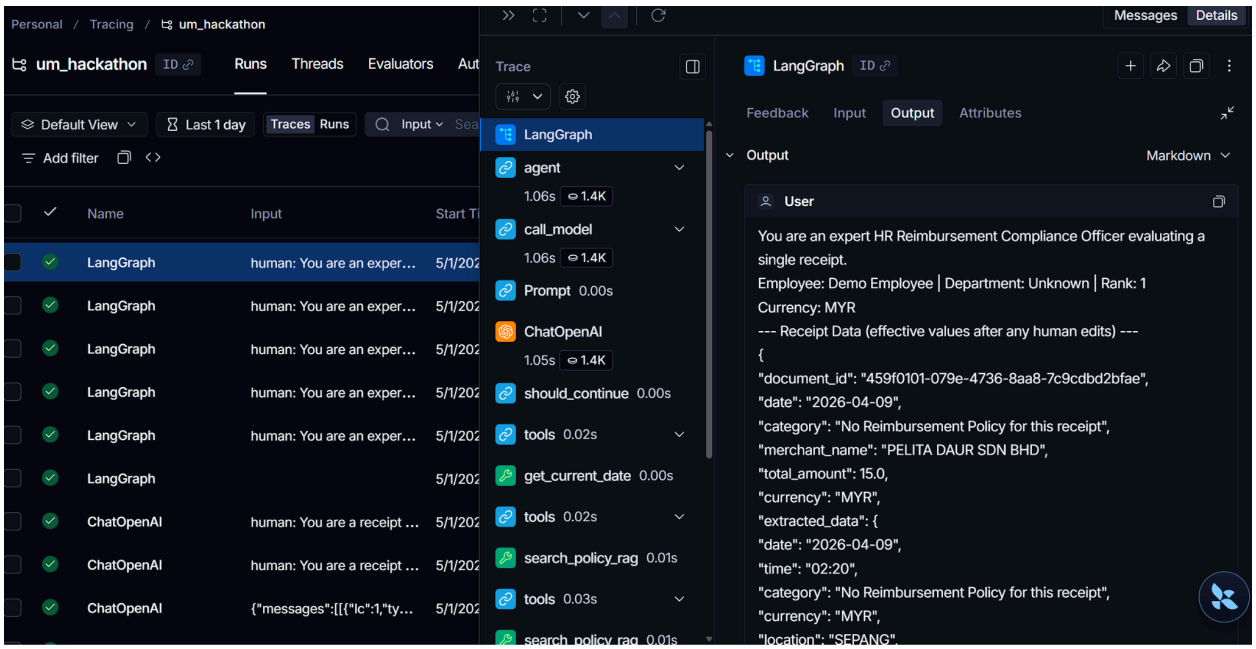
Engineering Rationale: Reclaim's AI pipeline makes calls to paid external APIs (OpenRouter for LLM and embedding inference) on every claim submission and policy upload. Without rate limiting, a single malicious actor or a runaway client script could spam these endpoints and exhaust the entire OpenRouter credit balance in minutes — taking the platform offline for all users. Rate limiting is therefore LLM budget protection as much as it is abuse prevention.

4.7 AI Observability via LangSmith

What We Built: LangSmith tracing is integrated into all three LangGraph pipelines (Policy Agent, Document Agent, Compliance Agent). When `LANGSMITH_TRACING_V2=true` is set, every agent run — including individual ReAct tool calls, intermediate reasoning steps, token counts, and latencies — is captured and visible in the LangSmith dashboard.

© UMHackathon 2026

Engineering Rationale: Traditional server monitoring (CPU, memory, HTTP status codes) is blind to AI-specific failure modes. A receipt-auditing agent could complete its HTTP call successfully from the server's perspective while simultaneously hallucinating a compliance judgment, exceeding its tool call budget, or entering a reasoning loop. LangSmith provides granular visibility into the *logic* of each agent execution, enabling us to catch and debug these failures at the prompt and reasoning level — not just at the infrastructure level.



Reclaim's Langsmith Dashboard Screenshot

5. Deployment Strategy

5.1 Methodology

Rolling container replacement via Dokploy. When a new image is built, Dokploy brings up the new container, waits for its health check to pass, then terminates the old container. This achieves zero-downtime updates for stateless services (frontend, backend).

The database container is not replaced during application deployments; schema changes are applied via Alembic migrations run inside the backend container after it starts.

© UMHackathon 2026

5.2 CI/CD Integration

Step	Mechanism
Code push to main	Triggers GitHub webhook to Dokploy
Dokploy webhook received	Pulls latest commit via read-only SSH Deploy Key
Image build	Dokploy runs `docker compose -f docker-compose.production.yml build`
Container replacement	Rolling update: new container starts → health check passes → old container stops
Migration	Alembic `upgrade head` runs automatically as part of backend container startup

The SSH Deploy Key stored in Dokploy has read-only (`--read-only`) repository access. It cannot push, create branches, or modify repository state.

6. Pre-Deployment Checklist

6.1 Infrastructure Readiness

- ☑ VPS has sufficient free memory for all three containers + Dokploy overhead (~350 MB for Dokploy, ~512 MB for Postgres buffer pool, backend headroom for parallel agent execution)
- ☑ Docker and Docker Compose versions are pinned and up to date on the VPS
- ☑ Traefik is running and SSL certificates are provisioned for the production domain
- ☑ Cloudflare DNS record is set to **Proxied** (orange cloud) — not DNS-only
- ☑ UFW is configured: ports 80/443 open **only to Cloudflare IP ranges**, all other direct traffic denied
- ☑ Dokploy Autodeploy webhook is active and points to the correct branch (`main`)

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.2 Secret Verification

- ☑ All backend environment variables listed in Section 3 are set in Dokploy's Environment UI
- ☑ `SECRET_KEY` is a cryptographically random string
- ☑ No `.env` files with real credentials exist in the repository

6.3 Data Preservation

- ☑ Manual snapshot of the `postgres_data` Docker volume taken before deployment
- ☐ Snapshot stored offsite or verified restorable (e.g., Cloudflare R2 bucket path)

6.4 Migration Testing

- ☑ Alembic migration scripts have been dry-run against a staging database matching the current production schema
- ☑ All migration files present in `backend/alembic/versions/` are accounted for

6.5 API Quota Check

- ☑ OpenRouter credit balance confirmed sufficient for expected production load
- ☑ LangSmith project exists and API key is valid
- ☑ Rate limiting middleware (`slowapi`) is deployed and configured on AI endpoints (`/api/v1/reimbursements`, `/api/v1/documents`, `/api/v1/policies`)

6.6 Code Readiness

- ☑ `WATCHFILES_FORCE_POLLING` is absent from the production compose file
- ☑ Seed scripts (`seed_demo_employees.py`, `seed_mock_policy.py`, `seed_mock_data.py`) are not in the production container startup command

© UMHackathon 2026

Email: umhackathon@um.edu.my

7. Future Architecture Roadmap

The following enhancements are deliberately deferred due to the time and budget constraints of the hackathon phase. Each is architected as a concrete next step for when Reclaim moves into a funded, scaling phase. They are not missing features — they are conscious trade-offs acknowledged upfront.

7.1 Distributed Redis-Based Rate Limiting

What: Deploy a centralized Redis cluster and replace the current in-memory slowapi rate limit counters with Redis-backed state shared across all backend container instances.

Why it matters: The current rate limiting tracks request counts in the memory of each individual container. This works correctly when only one backend container is running, but breaks under horizontal scale: if two backend replicas exist and a user sends five requests to each, they have effectively sent ten requests against a limit of five — the containers have no shared counter. As Reclaim scales out to multiple backend replicas (see 7.3), in-memory rate limiting becomes trivially bypassable. A centralized Redis store gives every replica a single source of truth for rate limit state, making enforcement globally consistent regardless of how many instances are running.

7.2 Automated PostgreSQL Volume Backups to S3/MinIO

What: Implement a scheduled job (e.g., a cron container using pg_dump) that exports the PostgreSQL database and streams the dump to an external S3-compatible object store such as Cloudflare R2 or a self-hosted MinIO instance. Backups would run on a defined cadence (e.g., daily) with retention policies.

Why it matters: Currently, all persistent data — employee records, claim submissions, extracted receipt data, policy embeddings — lives exclusively on the VPS's local disk volume. If the host provider experiences a catastrophic hardware failure, data corruption, or accidental volume deletion, every byte of production data is permanently lost with no recovery path. External backups decouple data durability from the health of a single physical server. For a

© UMHackathon 2026

reimbursement platform handling real financial records, this is not optional at scale — it is a baseline compliance requirement.

7.3 High Availability via Multi-Node Docker Swarm

What: Evolve the current single-VPS Dokploy deployment into a multi-node Docker Swarm cluster, distributing backend and frontend service replicas across at least three physical servers in geographically distinct availability zones. The database would be promoted to a managed, replicated PostgreSQL service (e.g., a primary + read replica configuration).

Why it matters: The current architecture has a single point of failure (SPOF): if the VPS's host hypervisor crashes, the network switch fails, or the data center loses power, the entire Reclaim platform goes offline instantly. There is no failover. For a hackathon this is an acceptable trade-off; for a production reimbursement platform processing real employee expense data, it is not. A multi-node Swarm distributes service replicas so that the failure of any single node triggers automatic rescheduling of its containers onto surviving nodes — eliminating the SPOF and providing the redundancy required for meaningful uptime SLAs.

7.4 Dynamic Secret Management via HashiCorp Vault

What: Migrate all API keys and credentials from Dokploy's internal environment store to HashiCorp Vault, configured to issue short-lived, dynamically generated credentials to the backend at runtime. The application would fetch secrets from Vault on startup rather than reading static environment variables.

Why it matters: The current model stores secrets as long-lived static strings in Dokploy's internal database, which lives on the same VPS as the application. If the VPS is fully compromised — root access gained by an attacker — all credentials are immediately readable from the environment. Vault solves this by issuing credentials with a short TTL (e.g., 1 hour). Even if an attacker captures a credential from a running process, it expires before they can weaponize it against the upstream API provider. Vault also provides a full audit log of every secret access event — a hard compliance requirement in regulated industries handling financial data.

© UMHackathon 2026

7.5 Self-Hosted Agent Tracing via Langfuse

What: Deploy a self-hosted instance of Langfuse (an open-source LLM observability platform) as a new Docker service within the Dokploy environment. We will reconfigure the LangGraph pipelines to route all agent telemetry to this local instance instead of LangSmith, storing all trace data, token usage, and execution steps directly into our existing PostgreSQL database using its native JSONB capabilities.

Why it matters: Currently, Reclaim relies on LangSmith's cloud service for visibility into agent reasoning. While powerful for prototyping, this introduces two major bottlenecks at scale: strict free-tier usage limits, and the transmission of highly sensitive execution contexts (raw financial receipts, internal company policies, employee PII) to a third-party server. By migrating to a self-hosted Langfuse architecture, Reclaim achieves complete data sovereignty. All prompts, intermediate reasoning steps, and tool calls remain strictly within our own network boundary. This ensures absolute privacy and regulatory compliance for financial data while permanently eliminating third-party SaaS caps and observability costs.

© UMHackathon 2026

Email: umhackathon@um.edu.my

Reclaim

Prepared for

UM Hackathon 2026
Judges

Prepared by

Team bogosort